

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

Анализ больших текстовых данных и текстовый поиск

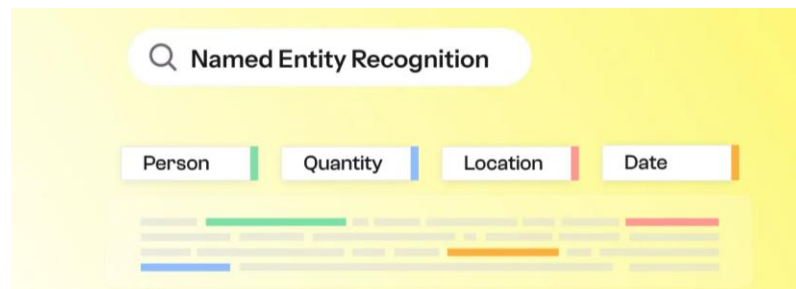
Лекция 8

20 октября 2025

Распознавание именованных сущностей (NER)

Именованная сущность (Named Entity) — это слово или словосочетание в тексте, обозначающее конкретный объект из реального мира, принадлежащий к определённой категории (например, человек, организация, место и т.д.)

Распознавание именованных сущностей (Named Entity Recognition, NER) — это задача обработки естественного языка (NLP), направленная на поиск, выделение и классификацию таких сущностей в тексте для последующего анализа и обработки.



Распознавание именованных сущностей (NER)

Имена собственные	
Метка	Описание
PERSON	имя человека (в том числе отдельно имя или фамилия, прозвище и т.п.)
ORG	наименование организации
GPE	административно-политическая единица (страна, регион, город и т.д.)
LOC	место, локация
FAC	объект инфраструктуры, имеющий название
PRODUCT	конкретный продукт с названием
LAW	конкретный закон с названием
WORK_OF_ART	конкретное произведение с названием
EVENT	событие, имеющее название

Распознавание именованных сущностей (NER)

Имена нарицательные	
Метка	Описание
TITLE	титул, должность или статус
JOB	наименования профессий
NORP	группа лиц, объединённых по признаку
BUSINESS	организационно-правовая форма или род деятельности организации
ARTEFACT	результат деятельности человека, имеющий лишь нарицательное название
LANGUAGE	наименование языка
MATERIAL	химические элементы и соединения, природные материалы, части целого
NATURAL	живые организмы, включая вирусы (исключая слишком общие понятия “человек”, “люди”)

Распознавание именованных сущностей (NER)

Числовые показатели	
Метка	Описание
DATE	абсолютная или относительная дата
TIME	время суток
ORDINAL	порядковое
QUANTITY	количество
PERCENT	процент, доля
MONEY	денежное выражение
CARDINAL	остальные числительные, например коэффициенты

Распознавание именованных сущностей (NER)

Компания Яндекс была основана в 1997 году Аркадием Воложем и Ильёй Сегаловичем в Москве

Сущность	Метка
Компания Яндекс	ORG
1997 году	DATE
Аркадием Воложем	PERSON
Ильёй Сегаловичем	PERSON
Москве	GPE

Распознавание именованных сущностей (NER)

Основные сценарии использования NER

- **Извлечение информации**

NER используется для поиска и систематизации данных, релевантных конкретным запросам. Например, в научных базах данных при вводе запроса «Новые методы диагностики рака легких» система определяет, что «рак легких» — это заболевание, а «методы диагностики» — область медицины. Это позволяет извлечь только статьи по соответствующей тематике, исключая нерелевантные материалы.

Распознавание именованных сущностей (NER)

- **Рекомендательные системы**

NER помогает платформам персонализировать выдачу контента. Так, если пользователь часто читает новости о возобновляемой энергетике и солнечных панелях, система с помощью NER выделяет эти сущности и предлагает похожие материалы — о новых технологиях или компаниях в этой сфере.

- **Автоматизация ввода данных**

В бизнес-среде NER применяется в системах роботизированной автоматизации процессов. Например, финансовая компания может использовать NER для обработки электронных счетов, автоматически извлекая из них название клиента, сумму платежа и дату выставления счета — избавляя сотрудников от ручного ввода данных.

Распознавание именованных сущностей (NER)

- **Уточнение анализа тональности**

Совместное использование NER и анализа эмоционального настроения позволяет выявлять, к каким объектам относятся положительные или отрицательные эмоции. Например, в отзыве «Сервис авиакомпании отличный, но регистрация на рейс заняла вечность» NER помогает определить, что положительная оценка связана с сервисом, а негативная — с процессом регистрации, что делает анализ отзывов значительно точнее.



Распознавание именованных сущностей (NER)

Роль NER в NLP-пайплайне

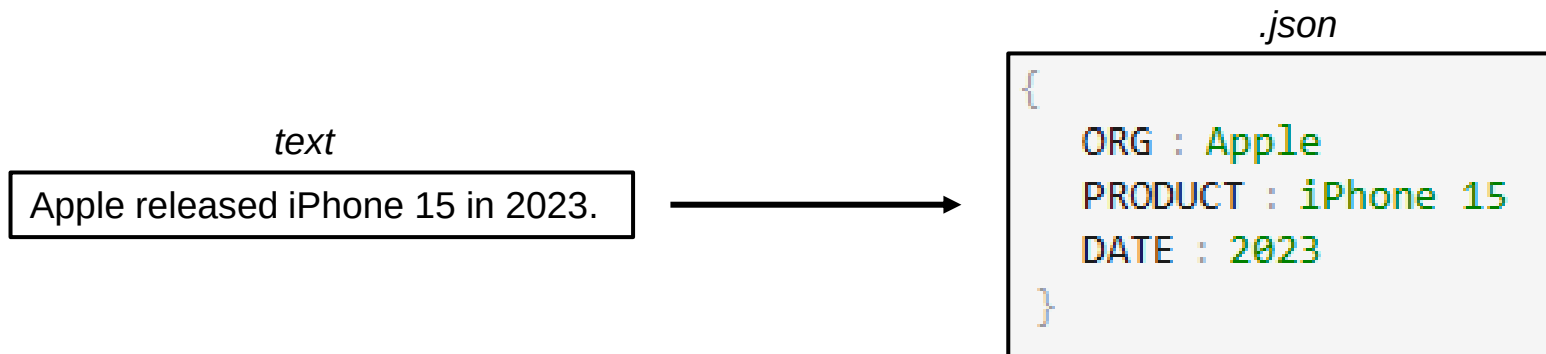
В реальных NLP-системах NER часто стоит после токенизации и морфологического анализа. Результаты NER могут дальше использоваться для семантического анализа и дальнейшего принятия решений (например, действий в диалоговых системах)



Распознавание именованных сущностей (NER)

Ключевая идея NER

Цель NER — превратить неструктурированный текст в структурированные данные



Постановка задачи

Задача NER

На вход подаётся предложение — последовательность слов (токенов):

$$x = [x_1, x_2, \dots, x_n]$$

На выходе — такая же последовательность меток:

$$y = [y_1, y_2, \dots, y_n]$$

Каждая метка указывает, к какой категории относится слово (или вне сущности)

Пример:

$x = ["Компания", "Яндекс", "была", "основана", "в", "1997", "году", "в", "Москве"]$

$y = ["O", "B-ORG", "O", "O", "O", "B-DATE", "I-DATE", "O", "B-GPE"]$

Разметка данных

BIO (Begin–Inside–Outside) схема разметки

Стандартный способ разметки сущностей в корпусах — **BIO**:

- B-XXX — начало сущности типа XXX
- I-XXX — продолжение сущности
- O — токен вне сущности

Токен	Разметка
Аркадий	B-PER
Волож	I-PER
основал	O
Яндекс	B-ORG
в	O
Москве	B-LOC

Разметка данных

BILOU — Begin, Inside, Last, Outside, Unit

Расширенная схема разметки. Позволяет различать однословные сущности (U) и последние токены (L)

Токен	Разметка
Аркадий	B-PER
Волож	L-PER
основал	O
Яндекс	U-ORG
в	O
Москве	U-LOC

В реальных проектах часто используется BILOU, так как она точнее кодирует границы и улучшает обучение

Последовательное моделирование

NER можно рассматривать как задачу **последовательного моделирования**:

$$P(y_1, y_2, \dots, y_n \mid x_1, x_2, \dots, x_n)$$

Где важно учитывать контекст. Метка y_i зависит не только от слова x_i , но и от соседних слов и их меток

Почему важен контекст:

Одинаковые слова могут быть разными сущностями или вообще не сущностями.

Поэтому NER требует анализа контекста, а не только словаря.

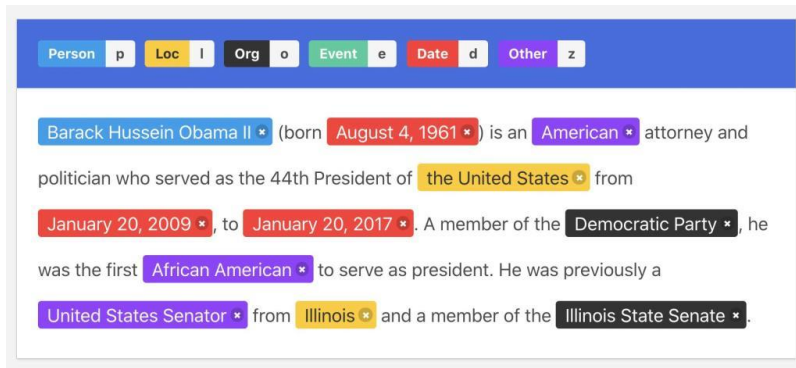
«**Волга** впадает в Каспийское море» — **LOC** (река)

«**Волга** стоит на парковке у дома» — **PRODUCT** (автомобиль)

Подходы к NER

Основные подходы к решению задачи NER:

- Правила и словари
- Статистические методы
- Классические ML-модели
- Нейросетевые модели



Правила и словари

Ранние системы NER строились на:

- Ручных правилах (регулярные выражения, шаблоны)
- Словарях / базах имён
- Грамматических шаблонах (например, «г-н+ Фамилия»)

Хотя сегодня такие методы считаются слишком простыми, они до сих пор полезны в данных с чёткой структурой — например, при извлечении реквизитов из договоров

ЗАКАЗЧИК:

ООО «Альфа»
улица Правды, дом 1, корпус 2,
Москва, 123456, Россия
Банковские реквизиты:
ИНН: 1111111111
КПП: 222222222
р/с: 40702310991119000856 в
Химкинском отделении № 7319
ОАО «Сбербанк России», г. Москва
Корр. сч: 30101810400000000711
БИК: 047501711

ИСПОЛНИТЕЛЬ:

ООО «Бета»
Проспект Мира, дом 30, оф. 345,
Москва, 123995, Россия
Банковские реквизиты:
ИНН 2222222222
КПП 111111111
л/с 20937Ц83140 в отделении по ЗАО
УФК по г. Москве
р/с 40501810600002000079
Отделение 1 Московского ГТУ Банка
России г. Москва 705
БИК 044583001 (корр. счета нет)

Правила и словари

Шаблоны

Пример шаблона на Python:

```
import re

pattern = r"[A-Z][a-z]+ [A-Z][a-z]+"
text = "Barack Obama met Angela Merkel in New York."
re.findall(pattern, text)

['Barack Obama', 'Angela Merkel', 'New York']
```

Такой шаблон находит пары слов, начинающиеся с заглавной буквы

Правила и словари

Словари и базы имен

Ещё один подход — использовать заранее собранные списки:

- Имена и фамилии людей
- Города и страны
- Организации, компании, бренды

Словари хороши для точных названий — например, городов или лекарств. Но они не помогут при опечатках, склонениях и новых именах, которые не включены в список.

Флаг	Страна	Столица
	Австрия	Вена
	Албания	Тирана
	Андорра	Андорра-ла-Велья
	Белоруссия	Минск
	Бельгия	Брюссель
	Болгария	София
	Босния и Герцеговина	Сараево
	Ватикан	Ватикан
	Великобритания	Лондон
	Венгрия	Будапешт
	Германия	Берлин
	Греция	Афины
	Дания	Копенгаген
	Ирландия	Дублин
	Исландия	Рейкьявик
	Испания	Мадрид
	Италия	Рим
	Латвия	Рига
	Литва	Вильнюс
	Лихтенштейн	Вадуц
	Люксембург	Люксембург
	Македония	Скопье
	Мальта	Валетта
	Молдавия	Кишинев
	Монако	Монако

Правила и словари

Преимущества	Недостатки
Простота реализации	Плохо работает на неструктурированных текстах
Не требует обучения	Не понимает контекста
Высокая точность в узких данных	Плохая обобщаемость, нет адаптации
Можно комбинировать с ML	Требует ручной поддержки

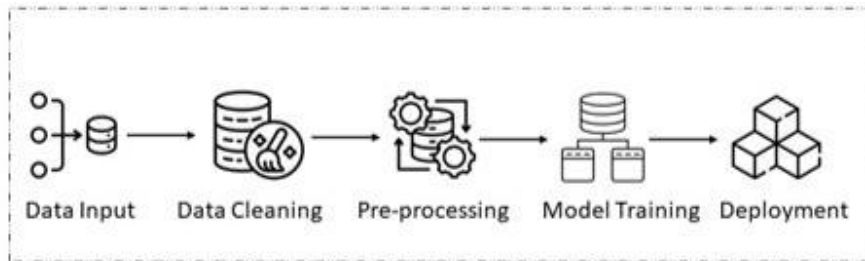
Главное достоинство — контроль и прозрачность: мы точно знаем, почему выделена сущность. Но если тексты разнообразны, правила начинают сыпаться

Статистические модели

Ручные правила и словари хорошо работают на простых текстах, но плохо масштабируются

- Много исключений
- Сложно поддерживать
- Появляются новые имена и формы

Решение: использовать **вероятностные модели**, которые учатся на размеченных данных



Скрытая марковская модель

Hidden Markov Model — это классическая вероятностная модель для последовательных данных. Мы считаем, что каждое слово в тексте порождено скрытым состоянием — меткой сущности, которую мы не видим напрямую. Например, если текущее слово имеет метку **B-PER**, то следующая метка с высокой вероятностью будет **I-PER**, а не **O**.

Таким образом, модель описывает два процесса:

- Переход между метками — как метки сменяют друг друга в последовательности (например, B-PER — I-PER — O)
- Эмиссию слов — какие слова обычно встречаются при каждой метке

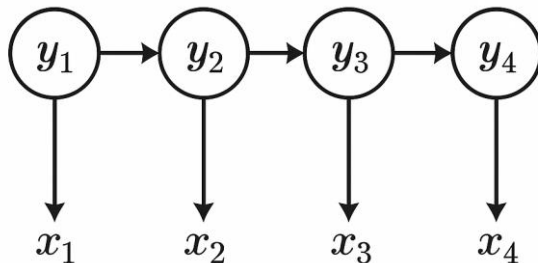
Скрытая марковская модель

- Каждое слово имеет скрытую метку (тип сущности)
- Последовательность меток образует марковскую цепь
- Модель описывает совместную вероятность слов и меток:

$$P(y_1, \dots, y_n, x_1, \dots, x_n) = \prod_i P(y_i \mid y_{i-1}) \cdot P(x_i \mid y_i)$$

$P(y_i \mid y_{i-1})$ — вероятность перехода между метками

$P(x_i \mid y_i)$ — вероятность получения слова из данной метки



Скрытая марковская модель

HMM работает с двумя типами вероятностей:

1. Переходы между метками $P(y_i | y_{i-1})$

Вероятность, что следующая метка будет y_i , если предыдущая была y_{i-1}

Пред. метка	След. метка	Вероятность
START	PERSON	0.6
START	O	0.4
PERSON	O	0.9
PERSON	PERSON	0.05
O	ORG	0.3
O	O	0.6
O	PERSON	0.1
ORG	O	1.0

Скрытая марковская модель

2. Эмиссии $P(x_i|y_i)$

Вероятность, что при метке y_i встретится слово x_i

Метка	Слово	Вероятность
PERSON	Анна	0.3
PERSON	работает	0.05
PERSON	Яндексе	0.02
O	работает	0.3
O	в	0.4
O	Яндексе	0.05
ORG	Яндексе	0.5
ORG	в	0.05

Скрытая марковская модель

Когда мы подаём новую фразу, например:

«Анна работает в Яндексе»

Модель не знает, какие метки стоят за словами. Она перебирает все возможные последовательности меток и выбирает ту, у которой вероятность $P(y,x)$ **максимальна**

Допустим, мы проверяем две гипотезы:

Вариант	Метки	Интерпретация
A	PERSON, O, O, ORG	Анна — человек, Яндекс — организация
B	O, O, O, O	модель не нашла сущностей

Скрытая марковская модель

«Анна работает в Яндексе»

$$P(y, x) = \prod_i P(y_i | y_{i-1}) \cdot P(x_i | y_i)$$

Вариант А: [PERSON, O, O, ORG]

Шаг	Что оцениваем	Значение
1	$P(y_1 = \text{PERSON} \mid \text{START}) \cdot P(\text{«Анна»} \mid \text{PERSON})$	$0.6 \times 0.3 = 0.18$
2	$P(y_2 = \text{O} \mid y_1 = \text{PERSON}) \cdot P(\text{«работает»} \mid \text{O})$	$0.9 \times 0.3 = 0.27$
3	$P(y_3 = \text{O} \mid y_2 = \text{O}) \cdot P(\text{«в»} \mid \text{O})$	$0.6 \times 0.4 = 0.24$
4	$P(y_4 = \text{ORG} \mid y_3 = \text{O}) \cdot P(\text{«Яндексе»} \mid \text{ORG})$	$0.3 \times 0.5 = 0.15$

$$P_A = 0.18 \times 0.27 \times 0.24 \times 0.15 = \mathbf{0.0017496} \approx \mathbf{0.00175}$$

Скрытая марковская модель

«Анна работает в Яндексе»

$$P(y, x) = \prod_i P(y_i | y_{i-1}) \cdot P(x_i | y_i)$$

Вариант В: [О, О, О, О]

Шаг	Что оцениваем	Значение
1	$P(y_1 = \text{О} \mid \text{START}) \cdot P(\text{«Анна»} \mid \text{О})$	$0.4 \times 0.01 = 0.004$
2	$P(y_2 = \text{О} \mid y_1 = \text{О}) \cdot P(\text{«работает»} \mid \text{О})$	$0.6 \times 0.3 = 0.18$
3	$P(y_3 = \text{О} \mid y_2 = \text{О}) \cdot P(\text{«в»} \mid \text{О})$	$0.6 \times 0.4 = 0.24$
4	$P(y_4 = \text{О} \mid y_3 = \text{О}) \cdot P(\text{«Яндексе»} \mid \text{О})$	$0.6 \times 0.05 = 0.03$

$$P_B = 0.004 \times 0.18 \times 0.24 \times 0.03 = \mathbf{0.000005184} = \mathbf{5.184 \times 10^{-6}}$$

Скрытая марковская модель

Преимущества метода:

№	Преимущество	Пояснение
1	Простая и интерпретируемая	Каждая вероятность имеет понятный смысл: переход или эмиссия. Можно легко объяснить, почему модель выбрала ту или иную метку
2	Формально строгая	Это полная вероятностная модель — она описывает весь процесс генерации слов из меток
3	Подходит для небольших данных	Можно обучить даже на малом размеченном корпусе — просто посчитать частоты

Скрытая марковская модель

Недостатки метода:

№	Недостаток	Пояснение
1	Сильные предположения	Марковское свойство: текущее состояние зависит только от предыдущего, не от всего контекста. Это ограничивает точность.
2	Независимость слов	HMM считает, что каждое слово зависит только от своей метки, а не от соседних слов, а это не так
3	Плохая работа с новыми словами	В простом случае используются OOV (Out Of Vocabulary) заглушки
4	Ограниченные признаки	Нельзя напрямую учитывать признаки вроде заглавных букв или суффиксов — только сами слова
5	Неустойчива к шуму	Опечатки, морфология и редкие формы ломают вероятности
6	Нет семантики	«Москва» и «Париж» считаются совершенно разными словами, даже если они того же типа

Conditional Random Fields

Conditional Random Fields (CRF, Условное случайное поле) — это дискриминативная вероятностная модель, определяющая распределение вероятностей меток $y=[y_1, y_2, \dots, y_n]$ при известных наблюдениях $x=[x_1, x_2, \dots, x_n]$

$$P(y|x) = \frac{1}{Z(x)} \exp \left(\sum_{i=1}^n \sum_j w_j f_j(y_i, y_{i-1}, x, i) \right)$$

CRF моделирует условное распределение меток, не делая предположений о том, как данные порождаются. Модель учится напрямую, какие метки наиболее вероятны для данной последовательности наблюдений, учитывая все признаки слов и связи между соседними метками

Conditional Random Fields

Основная формула CRF

$$P(y \mid x) = \frac{1}{Z(x)} \exp \left(\sum_{i=1}^n \sum_{j=1}^m w_j f_j(y_i, y_{i-1}, x, i) \right)$$

Символ	Значение	Комментарий
$x = [x_1, x_2, \dots, x_n]$	последовательность слов	например, "Илон основал Tesla в Калифорнии"
$y = [y_1, y_2, \dots, y_n]$	метки для слов	PERSON, O, ORG, O, LOC
$f(y_i, y_{i-1}, x, i)$	признаки (features)	бинарные или числовые функции, зависящие от слова, контекста и меток
w	веса модели	показывают, насколько каждый признак важен
$Z(x)$	нормирующий коэффициент	сумма по всем возможным последовательностям меток, чтобы вероятности суммировались до 1

Conditional Random Fields

Пример

«Илон основал Tesla в Калифорнии»

Мы хотим посчитать вероятность такой комбинации меток при таком тексте:

- Илон — **PERSON**
- основал — **O**
- Tesla — **ORG**
- в — **O**
- Калифорнии — **LOC**

Conditional Random Fields

1. Определим признаки (features)

Каждый токен описывается набором бинарных признаков — флагов, которые включаются (1), если условие выполняется, иначе 0

№	Признак	Обозначение
f_1	Слово с заглавной буквы	capital
f_2	Содержит латиницу	latin
f_3	Это глагол	verb
f_4	Предыдущее слово = “в”	after_v
f_5	Суффикс “-ии”	suf_ii
f_6	Предыдущая метка PERSON, а текущая O	t_per_o
f_7	Предыдущая метка O, а текущая ORG	t_o_org
f_8	Предыдущая метка O, а текущая LOC	t_o_loc

Conditional Random Fields

2. Признаки, активные для каждого токена

Слово	Метка	Активные признаки ($f_j=1$)
Илон	PERSON	capital
основал	O	verb, t_per_o
Tesla	ORG	capital, latin, t_o_org
в	O	—
Калифорнии	LOC	capital, suf_ii, after_v, t_o_loc

Conditional Random Fields

3. Присвоим веса признакам

Признак	Обозначение	Вес (w_j)	Комментарий
capital to PERSON	w_1	+2.0	Заглавная буква = имя
latin to ORG	w_2	+1.5	Латиница характерна для брендов
verb to O	w_3	+1.2	Глаголы вне сущностей
after_v to LOC	w_4	+1.0	После “в” часто идёт локация
suf_ii to LOC	w_5	+1.0	Конец “-ии” типичен для топонимов
t_per_o	w_6	+0.8	После PERSON обычно O
t_o_org	w_7	+1.0	После O часто ORG
t_o_loc	w_8	+1.0	После O часто LOC

Conditional Random Fields

4. Подставим активные признаки и посчитаем оценку согласованности (score)

$$S(y, x) = \sum_{i=1}^n \sum_{j=1}^m w_j f_j(y_i, y_{i-1}, x, i)$$

Слово	Активные признаки	Вклад (w_j)	Сумма
Илон	capital to PERSON	+2.0	2.0
основал	verb to O, t_per_o	+1.2 + 0.8	2.0
Tesla	capital to ORG, latin to ORG, t_o_org	+2.0 + 1.5 + 1.0	4.5
в	—	0	0.0
Калифорнии	capital to LOC, suf_ii to LOC, after_v to LOC, t_o_loc	+2.0 + 1.0 + 1.0 + 1.0	5.0

$$S(y, x) = 2.0 + 2.0 + 4.5 + 0.0 + 5.0 = 13.5$$

Conditional Random Fields

5. Считаем вероятность

$$S(y, x) = 13.5$$

Это **оценка согласованности (score)** между наблюдениями и выбранной последовательностью меток. Чем выше **$S(y, x)$** , тем лучше признаки и переходы поддерживают эту последовательность

Чтобы превратить это в вероятность:

$$P(y|x) = \frac{e^{S(y,x)}}{Z(x)}$$

$Z(x)$ — это сумма экспонент всех возможных оценок $S(y, x)$ для данного входа x

Conditional Random Fields

Преимущества

- Учитывает контекст последовательности. В отличие от простых классификаторов, CRF предсказывает всю цепочку меток сразу, а не каждое слово независимо. Это делает разметку более согласованной
- Использует гибкие признаки. Можно включать любые функции от входных данных x : суффиксы, часть речи, соседние слова, длину и т.д.
- Интерпретируемость. Веса w_j показывают, какие признаки и переходы важны. Можно анализировать, почему модель приняла то или иное решение. Хорошая совместимость с нейросетями
- CRF часто используется как финальный слой в BiLSTM-CRF и BERT-CRF, обеспечивая правильные переходы между метками

Conditional Random Fields

Недостатки

- Требует ручного проектирования признаков. Нужно самому придумать feature templates. Это трудоёмко и требует лингвистической интуиции
- Может быть вычислительно дорогой. Обучение требует вычисления $Z(x)$, что увеличивает время и память, особенно при большом числе меток
- Не учитывает дальние зависимости между метками. CRF видит только соседние метки (y_{i-1} , y_i). Для длинных связей нужно добавлять признаки вручную или использовать рекуррентные сети
- Чувствителен к качеству разметки и признаков. Если данные шумные, а признаки плохо выбраны — модель быстро деградирует. В отличие от BERT, CRF не извлекает информацию самостоятельно

Классические ML-подходы

Идея: для каждого слова можно вычислить вектор признаков и обучить классификатор, который предсказывает метку (PER, ORG, LOC и т.д.)

Модели:

- Логистическая регрессия
- SVM
- Random Forest
- Naive Bayes



Классические ML-подходы

Признаки (features):

- Форма слова — содержит ли цифры, дефис и т.п.
- Часть речи (POS-тег) — имя, существительное, глагол
- Контекст — предыдущее и следующее слово
- Префиксы и суффиксы — например, «-ов», «-ская», «ООО»
- Капитализация — начинается ли с большой буквы
- Длина слова, наличие символов (точка, кавычки и т.д.)

Признаки можно придумывать свои — любые логические или числовые функции, которые могут помочь отличить тип сущности

Признаки должны быть: дешево вычисляемыми (без внешних API), детерминированными (всегда одинаковый результат), зависимыми только от входных данных x (не от меток)

Классические ML-подходы

Пример признаков

Токен	Заглавная	POS	Пред. слово	След. слово	Метка
Москва	True	PROPER NOUN	в	—	LOC
ООО	True	NOUN	—	«Ромашка»	ORG
Сергей	True	PROPER NOUN	—	живёт	PER
банк	False	NOUN	в	России	ORG

Каждая строка — это один пример для обучения модели

Классические ML-подходы

Логистическая регрессия

Модель обучается предсказывать вероятность метки y_i по признакам $\mathbf{f}(x_i)$:

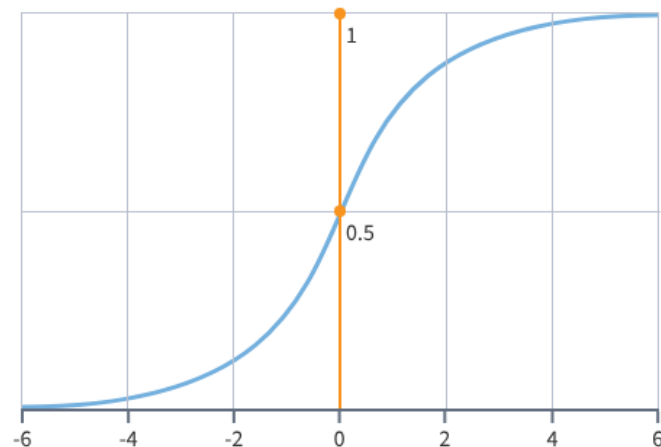
$$P(y_i = k \mid x_i) = \sigma(w_k^T \cdot f(x_i))$$

$f(x_i)$ — вектор признаков слова

w_k — веса для класса k

$\sigma(z) = \frac{1}{1+e^{-z}}$ — сигмоида (для бинарной регрессии)

или softmax для многоклассового случая

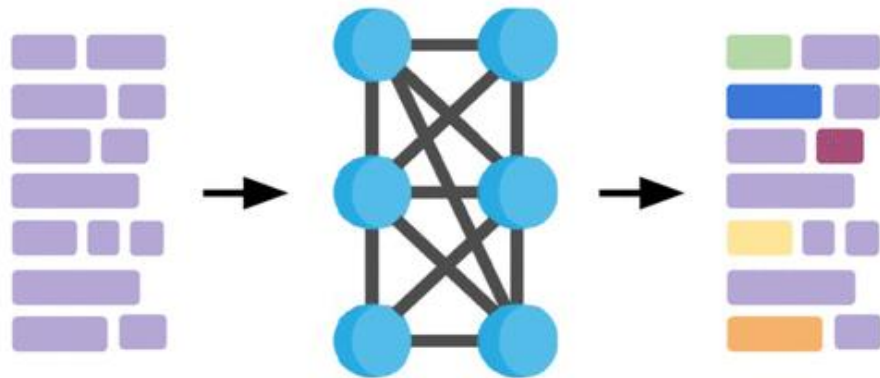


Нейросетевые подходы

Современные модели для Named Entity Recognition используют **нейросетевые архитектуры**, которые автоматически извлекают признаки из текста, вместо ручного проектирования feature templates

Основные архитектуры нейросетей:

- BiLSTM + CRF
- CNN + BiLSTM
- Трансформеры (BERT, GPT)

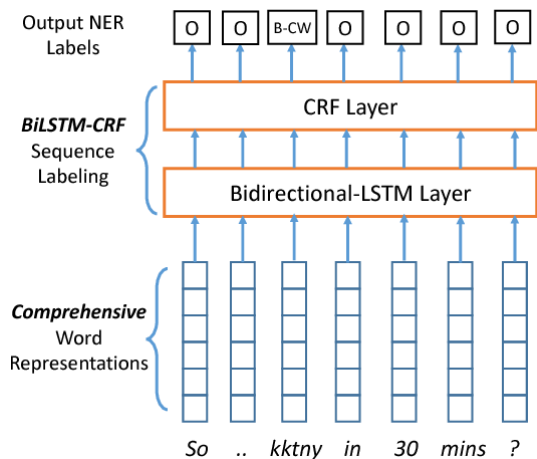


Нейросетевые подходы

BiLSTM + CRF

Идея:

- **BiLSTM** считывает предложение в обе стороны (вперёд и назад), создавая контекстные представления токенов
- **CRF** сверху обеспечивает глобально согласованную последовательность меток



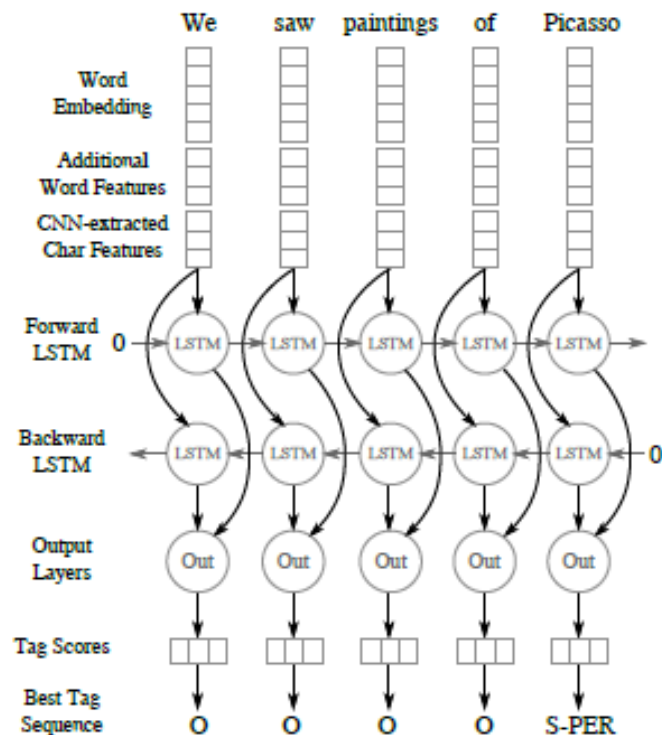
- Учитывает контекст до и после слова
- CRF корректирует локальные ошибки (например, не допускает I-PER после B-ORG)
- Работает даже без заранее заданных признаков

Нейросетевые подходы

CNN + BiLSTM

Идея:

- **CNN** извлекает символьные признаки (морфологию, суффиксы, префиксы), особенно полезно для редких и несловарных слов. По символам учит внутреннюю структуру слова (например, «-ов», «-ская», «ООО»)
- **BiLSTM** даёт контекст на уровне слов (использует контекст предложения)

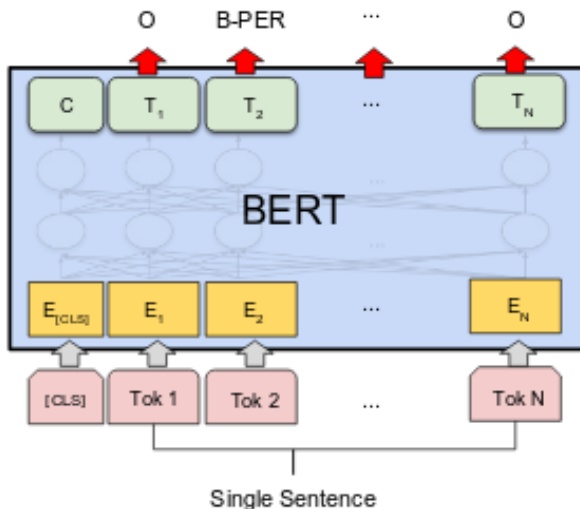


Нейросетевые подходы

Трансформеры (BERT, GPT)

Идея:

Трансформеры используют механизм внимания (**Self-Attention**), который учитывает весь контекст предложения сразу, а не пошагово, как LSTM



- Каждое слово получает контекстный embedding, зависящий от всех других слов в предложении
- Модель уже обучена на больших корпусах (например, Wikipedia)
- Для NER требуется только дообучение (fine-tuning) на размеченных данных

Метрики качества

Основные метрики

Метрика	Формула	Что показывает
Accuracy	$((TP + TN) / (TP + TN + FP + FN))$	Доля правильно классифицированных примеров
Precision (точность)	$(TP / (TP + FP))$	Насколько предсказанные положительные классы действительно положительные
Recall (полнота)	$(TP / (TP + FN))$	Какую долю реальных положительных классов модель нашла
F1-score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$	Баланс между точностью и полнотой

Token-level и entity-level evaluation

Token-level evaluation — сравнение меток по каждому токenu отдельно (считает, сколько слов размечено правильно)

Entity-level evaluation — сравнение целиком сущностей (границы + тип должны совпасть полностью)

Подход	Что считается	Проблема
Token-level	Каждое слово отдельно	Частично правильные сущности считаются верными
Entity-level	Сущность целиком (BIO / BIOES-style)	Требует полного совпадения границ и типа

Token-level и entity-level evaluation

«Глава компании Tesla Motors Илон Маск посетил Лос-Анджелес»

Истинные значения

Токен	Метка
Глава	O
компании	O
Tesla	B-ORG
Motors	I-ORG
Илон	B-PER
Маск	I-PER
посетил	O
Лос-Анджелес	B-LOC

Предсказания модели

Токен	Метка
Глава	O
компании	B-ORG
Tesla	I-ORG
Motors	O
Илон	B-PER
Маск	B-PER
посетил	O
Лос-Анджелес	I-LOC

Token-level и entity-level evaluation

«Глава компании Tesla Motors Илон Маск посетил Лос-Анджелес»

Token-level

Токен	ИСТИНА	Предсказано	Результат
Глава	O	O	True
компании	O	B-ORG	False
Tesla	B-ORG	I-ORG	False
Motors	I-ORG	O	False
Илон	B-PER	B-PER	True
Маск	I-PER	B-PER	False
посетил	O	O	True
Лос-Анджелес	B-LOC	I-LOC	False

Precision = 0.38, Recall = 0.38, $F_1 = 0.38$

Entity-level

Истинные сущности:

Tesla Motors — **ORG**

Илон Маск — **PERSON**

Лос-Анджелес — **LOC**

Предсказанные сущности:

компании Tesla — **ORG** (не те границы)

Илон — **PER** (укорочено)

Маск — **PER** (раздвоено)

Лос-Анджелес — **LOC** (неверное начало, I вместо B)

Precision = 0.0, Recall = 0.0, $F_1 = 0.0$